



**POLITECNICO
MILANO 1863**

**SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE**

REPORT OF THE PROJECT

Particle Swarm Optimization - Hands On Extension

LAUREA MAGISTRALE IN HIGH PERFORMANCE COMPUTING ENGINEERING - INGEGNERIA DEL CALCOLO AD ALTE PRESTAZIONI

Authors: MANTOAN, MESHCHERIAKOV, OGGIONI

Advisor: PROF. LUCA FORMAGGIA

Academic year: 2025-2026

Contents

1	Introduction	2	5	Conclusions	16
			5.1	Future work	17
2	Development	2	6	Acknowledgements	17
3	Implemented algorithms	2	7	Appendix	18
3.1	CHOPSO with arbitrary non-linear constraints	3			
3.2	Simulated Annealing PSO with arbitrary non-linear constraints .	5			
3.2.1	Implementation Details and Object-Oriented Design	6			
3.3	ELPSO with arbitrary non-linear constraints	6			
3.4	Handling constraints	7			
3.5	CD Pipeline	8			
3.6	Unit tests	8			
4	Benchmarks and comparisons	9			
4.1	Speed benchmarks	9			
4.1.1	CHOPSO benchmarks . .	9			
4.1.2	ELPSO benchmarks . . .	10			
4.1.3	Simulated Annealing benchmarks	11			
4.2	Convergence tests	13			
4.2.1	CHOPSO convergence . .	14			
4.2.2	SAOptimizer convergence	15			
4.2.3	GENETICOptimizer convergence	16			

1. Introduction

Particle Swarm Optimization (PSO) algorithms [2] are a kind of iterative algorithms used to approximate the global optimum for given functions. As the name implies, PSO algorithms are usually based on a swarm composed by moving particles that explore the global search space in order to find a *good enough* spot that approximate the global optimum.

There exist multiple variants of PSO algorithm, with new ones being suggested in the literature frequently: they differ in the rule that updates the particle positions (normal, chaotic, etc.) or in the specific optimization goals they address (single-objective, multi-objective, binary-objective, etc.).

Single-objective PSO algorithms work on a really broad range of functions: the only requirement is that the functions to be optimized attain at least one global maximum (maximizing problem) and at least one global minimum (minimizing problem). Due to the *explorative* nature of the algorithm, it does not require to know the gradient of the function to optimize (derivative-free optimization), therefore the gradient can also be undefined (e.g. in the case of a discontinuous function) and the algorithm would still work.

It's important to note that PSO algorithms are purely heuristic approach: mathematically speaking, particles could get stuck in a local best or not converge to anything at all. Despite this, in the literature, it has been shown that PSO algorithms are still a good choice. The robustness of the algorithm depends on the number of particles (the higher the better), on the various parameters (that depend on the *shape* of the objective function) and on the size and dimensionality of the global search space (the smaller the better). Nevertheless, overall efficiency is strictly dependent on the stopping criteria. These must be calibrated to minimize the accuracy-performance trade-off, often relying on a maximum iteration threshold as the primary termination condition.

In this project, we started from the *Hands On* results (CHOPSO - a chaotic variant of the plain PSO algorithm - with a limited box-shaped domain), extending it in order to get to an easy-to-

extend, easy-to-use, *header-only*, dependency-less (with optional support for OpenMP) C++ library, that comprises three different algorithms, namely

- CHOPSO with arbitrary non-linear constraints;
- Simulated Annealing optimization with arbitrary non-linear constraints;
- ELPSO with arbitrary non-linear constraints.

The entire source code of the project, can be found on [Github](#).

Benchmarks and comparisons between the different algorithms were also performed, in order to see which algorithms converges faster in which conditions and how they scale with respect to the number of processors.

In order to make results reproducible, the source code of the benchmarks is included in the same repository.

All the project has been documented with Doxygen. Documentation is available at <https://etabeta1.github.io/particle-swarm-optimization/>.

2. Development

The entire project spans across multiple C++ header files, multiple source files (unit tests and benchmarks), a single `Makefile` and a CD pipeline in form of a *GitHub Actions* workflow to automatically build and publish the Doxygen documentation.

Table 1 describes the structure, in terms of files and folders, of the entire project.

Table 2 goes over all the defined classes, in alphabetical order, it's a general overview on the project.

Figure 21 represent the classdiagram for the entire project.

3. Implemented algorithms

The aim of this section is to describe the various algorithms implemented in the project along with all the common support classes and methods shared by all of them.

Generally speaking, all the implemented algo-

Project structure

Folder	Content
<code>.github</code>	Folder containing the configuration for the GitHub Actions workflow.
<code>app</code>	Folder containing all the sources used to benchmark the project.
<code>benchmark</code>	Folder containing all the source code for the benchmarks.
<code>docs</code>	Folder in which rendered documentation will be placed by Doxygen.
<code>graphics</code>	Folder containing all the plots used to compare the various algorithms. All the plots on this file can be found there.
<code>json</code>	Git submodule which contains a json library used by the benchmarks.
<code>src</code>	Folder containing the library developed during the project.
<code>tests</code>	Folder containing the unit tests.
<code>tools</code>	Folder containing utility scripts used to parse and plot results.
<code>cluster.sh</code>	Script used to launch the benchmarks on the cluster
<code>Doxyfile</code>	Doxygen configuration
<code>Makefile</code>	Build system

Table 1: Folder structure of the project

gorithms follow more or less the same high level flow depicted in Figure 1.

The main characteristic that varies between the various algorithm is the way the particle update rule but the entire update loop is fully parallelized using OpenMP `#pragma omp parallel for`, employing the custom `findBestPoint` reduction to safely aggregate the global optimum across threads without incurring in race conditions or locking overheads for all the algorithms.

3.1. CHOPSO with arbitrary non-linear constraints

The CHOPSO algorithm [3] [2] is a variation of the conventional PSO algorithm in which, in addition to the normally moving particles, a number of chaotically moving particles are inserted in order to dramatically increase the speed at which the global search space is explored in order to achieve faster convergence and greater robustness against local optima.

According to the algorithm, the normal particles move following the update rule depicted in Equation 1.

$$\begin{cases} \mathbf{v}_i^{(k+1)} &= (0.9 - (0.5 * k/K)) \cdot \mathbf{v}_i^{(k)} \\ &+ \text{Rand}[0, 1] \cdot c_1 \cdot (\mathbf{P}_i - \mathbf{x}_i^{(k)}) \\ &+ \text{Rand}[0, 1] \cdot c_2 \cdot (\mathbf{G} - \mathbf{x}_i^{(k)}) \\ \mathbf{x}_i^{(k+1)} &= \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)} \end{cases} \quad (1)$$

where i is the particle index, k is the current iteration, K is the maximum number of iterations, c_1 is the cognitive coefficient (i.e. how much each particle trusts the best solution found by itself), c_2 is the social coefficient (i.e. how much each particle trusts the best solution found by the entire swarm), $\mathbf{P}_i$ is the best solution found by the i -th particle and $\mathbf{G}$ is the best solution found by the entire swarm.

At each iteration, each particle compare the previously found personal best solution and, if it worse than the current position, it updates it. After that, the global best solution is updated based of the best solution found by each particle.

Technical documentation for normal particles can be found [here](#).

Various factors must be taken into consideration when choosing the c_1 and c_2 parameters:

Class list

Class name	Description
AckleyFunction	Represents the <i>Ackley</i> function, one of the sample objective functions available in the project.
Alpine1Function	Represents the <i>Alpine1</i> function, one of the sample objective functions available in the project.
ChaosMap	Manages the evolution of the position of chaotic particles.
ChaoticParticle	Represents chaotic particles.
Chebyshev	Represents the <i>Chebyshev</i> chaotic map, one of the sample chaotic maps available in the project.
CHOPSOOptimizer	Contains all the logic used to implement the CHOPSO algorithm.
DropwaveFunction	Represents the <i>Dropwave</i> function, one of the sample objective functions available in the project.
EllipsoidFunction	Represents the <i>Ellipsoid</i> function, one of the sample objective functions available in the project.
EvaluatedPoint	Represents a tuple of a position and the value of the objective function evaluated at that position.
GENETICOOptimizer	Contains all the logic used to implement the ELPSO algorithm.
SAOptimizer	Contains all the logic used to implement the SAOptimizer algorithm.
Iterative	Represents the <i>Iterative</i> chaotic map, one of the sample chaotic maps available in the project.
LogisticMap	Represents the <i>LogisticMap</i> chaotic map, one of the sample chaotic maps available in the project.
NormalParticle	Represents normally moving particles.
ObjectiveFunction	Represents an objective function to be minimized.
Particle	Base class for all the particles.
Point	Represents a generic vector with most of the vector operations implemented.
QuinticFunction	Represents the <i>Quintic</i> function, one of the sample objective functions available in the project.
Sine	Represents the <i>Sine</i> chaotic map, one of the sample chaotic maps available in the project.
Singer	Represents the <i>Singer</i> chaotic map, one of the sample chaotic maps available in the project.
Sinusoidal	Represents the <i>Sinusoidal</i> chaotic map, one of the sample chaotic maps available in the project.
SphereFunction	Represents the <i>Sphere</i> function, one of the sample objective functions available in the project.

Table 2: List of the project classes

if c_1 is too big compared to c_2 , each particle will be more likely to get stuck in the first local optimum it find, while in the other case, when c_2 is too big compared to c_1 , the entire swarm is more likely to get stuck in a random local minimum without exploring much of the global

search space. According to both the literature[4] and our experience, setting both values between in the range $[1.5, 2.5]$ seems to provide good results (both in terms of particles not getting stuck anywhere else than in the global minimum and in convergence rate).

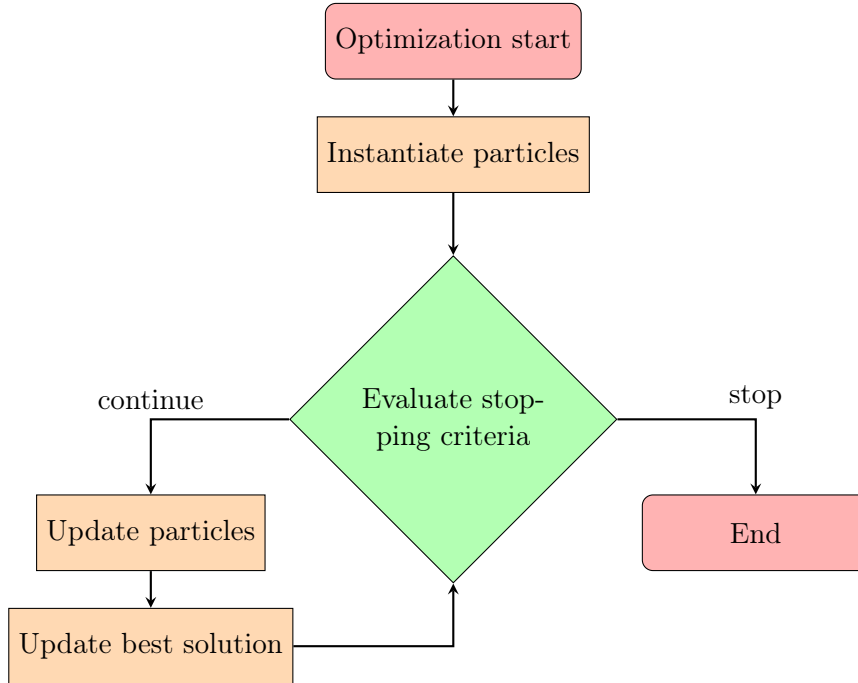


Figure 1: High level flow for all the algorithms implemented

The movement of the chaotic particles is based on chaotic maps that, given the current position and the current iteration number, return a new position inside the bounding box that encloses the global search space. Multiple chaotic maps were already reported in the literature [2] but as suggested by the simulations the Chebyshev one seems to provide the best results.

Technical documentation for chaotic particles and chaotic maps can be found [here](#) and [here](#), respectively.

Since chaotic maps need to know the boundaries of the search space for each dimension, one of the limit of the CHOPSO algorithm, is that the global search space must be confined by a box-shaped constraint.

The entire optimizer has been packed into an handy class (`CHOPSOoptimizer`) for easy instantiation and usage. Full API documentation can be found [here](#).

Support for parallelization of the `CHOPSOoptimizer` is available through the use of `OpenMP`: it is enough to compile with the `-fopenmp` flag and, optionally, to limit the number of threads calling `omp_set_num_threads` before starting the optimizer.

The way parallelization has been implemented,

is that in a single iteration, up to all the particles (assuming we have enough cores available) can be updated simultaneously and, after that, with a custom `OpenMP` reduction, the global best is updated.

Since it is possible to log the position of each particle to a file, and since the write to that file happens in the parallelized loop, a critical section is used to prevent race conditions. Obviously, this deteriorates greatly the performance of the program so it is advised to disable this functionality while performing speed benchmarks. Since our optimizer is very generic and can be adapted to multiple data types and dimensionality, the custom `OpenMP` reduction must be enabled with the `ENABLE_REDUCTION` macro, specifying the data type and the dimensionality of the problem.

3.2. Simulated Annealing PSO with arbitrary non-linear constraints

To mitigate the risk of premature convergence to local minima, which is a known vulnerability of the standard Particle Swarm Optimization algorithm, we implemented an hybrid variant incorporating the Simulated Annealing (SA) meta-heuristic. This approach, denoted as SA-PSO, modifies the standard position update rule by introducing a probabilistic acceptance criterion

based on thermodynamic principles.

From an algorithmic perspective, the standard velocity update relying on cognitive and social coefficients is preserved. However, the candidate position

$$\mathbf{x}_{\text{candidate}} = \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)}$$

is evaluated against the objective function $f(\mathbf{x})$ before the particle officially moves. If the candidate position yields a better or equal fitness value, meaning $f(\mathbf{x}_{\text{candidate}}) \leq f(\mathbf{x}_i^{(k)})$, the move is strictly accepted.

Conversely, if the candidate position is worse, it is not immediately discarded; instead, it is accepted with a probability P : a uniformly distributed random number $r \in [0, 1]$ is generated, and the move is accepted if $r < P$.

P is determined by the Boltzmann distribution:

$$P = \exp\left(\frac{f(\mathbf{x}_i^{(k)}) - f(\mathbf{x}_{\text{candidate}})}{T_k}\right)$$

where T_k is the current temperature of the system at iteration k .

3.2.1 Implementation Details and Object-Oriented Design

To seamlessly integrate this logic into our existing C++ template architecture, we leveraged inheritance and runtime polymorphism. The SA-PSO behavior is encapsulated within the `SANormalParticle<T, dim>` class, which publicly derives from the `NormalParticle<T, dim>` class. This design choice allows the SA particle to reuse the standard CHOPSO velocity update mechanism (`updateSpeed`) while overriding the virtual `updatePosition` method to inject the probabilistic acceptance logic.

The entire optimizer has been packed into an handy class (`SAOptimizer`) for easy instantiation and usage. Full API documentation can be found [here](#).

A key architectural requirement of this hybrid approach is that the `SANormalParticle` must evaluate candidate positions internally during the update phase, independently from the global swarm evaluation.

To achieve this without breaking encapsulation, the particle is injected with a `std::shared_ptr<ObjectiveFunction<T, dim>` upon instantiation. This allows the particle to safely invoke `evaluate()` on the candidate position locally.

At the swarm management level, the `SAOptimizer<T, dim>` orchestrates the optimization process. It introduces a geometric cooling schedule for the temperature. Because time equates to the progression of iterations, the temperature is updated precisely at the end of each iteration loop as $T_{k+1} = T_k \cdot \alpha$, where α is defined as the `cooling_rate`. Because the optimizer manages a heterogeneous collection of particles via base class pointers (`std::vector<std::unique_ptr<Particle<T, dim>>`), propagating the new temperature requires runtime type identification. We utilize a `dynamic_cast<SANormalParticle<T, dim>*>` to downcast and identify SA particles within the swarm, invoking their specific `setTemperature` method while leaving regular chaotic particles unaffected.

3.3. ELPSO with arbitrary non-linear constraints

ELPSO (Enhanced Leader Particle Swarm Optimization) is another kind of particle swarm optimization algorithm that further increases space exploration in order to minimize the probability of getting stuck in a local optimum in advance[5].

We implemented a system of random Mutations, that acts on global best solution, exploring different direction variation. In fact, our ELPSO Optimizer incorporates 5 different type of Mutations: **Uniform**, **Gaussian**, **Non Uniform**, **Dimension Wise** and **Opposition based**.

- **Uniform mutation** performs 1d coordinate mutation using uniform distribution on a randomly chosen coordinate of the global best.
Let $[a_x, b_x]$ be the search interval (bound) defined for the x -th coordinate. A new value v is generated such that:

$$\mathbf{v} \sim \mathcal{U}(\mathbf{a}_x, \mathbf{b}_x)$$

Where $\mathcal{U}$ represents a continuous uniform distribution within the specified interval.

Let $\mathbf{G} = (p_0, p_1, \dots, p_x, \dots, p_{dim-1})$ be the position vector of the *Global Best*. The new point $\mathbf{P}'$ is generated by replacing only the x -th component:

$$\mathbf{P}' = (p_0, p_1, \dots, v, \dots, p_{dim-1})$$

- **Gaussian mutation** performs a Gaussian perturbation on a randomly chosen coordinate of the global best.

Let $[a_x, b_x]$ be the search interval defined for the x -th coordinate. The standard deviation σ is calculated as:

$$\sigma = 0.05 \cdot (b_x - a_x)$$

A new value v is sampled from a normal distribution centered at the current coordinate p_x :

$$v \sim \mathcal{N}(p_x, \sigma^2)$$

- **Non Uniform mutation** performs a time-dependent non-uniform mutation on one coordinate of global best. Mutation magnitude decreases with the number of iterations to encourage exploitation at earlier stages.

$$v = \begin{cases} p_x + \Delta(t, b_x - p_x) & \text{if } \mathcal{U}(0, 1) < 0.5 \\ p_x - \Delta(t, p_x - a_x) & \text{otherwise} \end{cases}$$

The adaptive step size $\Delta(t, y)$ is defined by the following function:

$$\Delta(t, y) = y \cdot \left(1 - r^{(1 - \frac{t}{T})^b}\right)$$

where:

- $r \sim \mathcal{U}(0, 1)$ is a uniform random number.
- $b = 5.0$ is the shape parameter determining the degree of non-uniformity.
- y represents the maximum possible distance to the boundary ($b_x - p_x$ or $p_x - a_x$).
- **Dimension wise mutation** mutates each dimension of global best independently with small Gaussian. The mutation applies an independent Gaussian perturbation to each coordinate of the global best point ($\mathbf{G}$) based on a mutation probability p_m . The probability of mutating each individual dimension is defined inversely to the dimensionality of the space:

$$p_m = \frac{1}{\dim}$$

For each coordinate $i \in \{0, \dots, dim - 1\}$, a random value $u_i \sim \mathcal{U}(0, 1)$ is generated. If $u_i < p_m$, the coordinate p_i is updated like

$$v_i = \max(a_i, \min(b_i, \mathcal{N}(p_i, \sigma_i^2)))$$

where the standard deviation σ_i is equal to 5% of the search range for every dimension accordingly:

$$\sigma_i = 0.05 \cdot (b_i - a_i)$$

- **Opposition based mutation** takes each coordinate of a global best and performs reflection of each coordinate around the center of admissible region.

For each dimension $i \in \{0, \dots, dim - 1\}$, the coordinate p_i is transformed into its opposite coordinate v_i using the formula:

$$v_i = a_i + b_i - p_i$$

where a_i and b_i represent the lower and upper bounds of the i -th dimension, respectively.

After each iteration of the algorithm, we compare the actual global best with 5 different Mutations and choose the best one according to optimization function.

It is worth to mention that Mutations doesn't guarantee to find a solution better than the Global best found initially.

To handle all the logic behind this, we created `GENETICOptimizer<T, dim>`.

Since `GENETICOptimizer` inherits from `CHOPS0Optimizer`, then, it is possible to make `GENETICOptimizer` use also chaotic particles.

The entire optimizer has been packed into an handy class (`GENETICOptimizer`) for easy instantiation and usage. Full API documentation can be found [here](#).

3.4. Handling constraints

Additionally, our implementation of the three aforementioned algorithms support arbitrarily-shaped constraints. To give an example of how arbitrary the constraints our implementations support, we cite "The solution must be outside

of this polyhedron, except on the third day of each month: in that case it must be inside". The only limitation on the shape of the constraints is that their evaluation must not produce any side effect (in case this rule is not respected, undefined behavior may emerge).

The handling of the nonlinear constraints follows this rule: the personal best solution found by each particle can be update only if all constraints are respected. This allows us to not compute the value of the objective function outside of the boundaries (allowing us to also optimize on functions that may not be defined everywhere) while keeping the current best solution inside the boundaries. In order to enforce that the optimization problem must have a solution, an initial feasible solution must be given when instantiating the optimizer.

Since we never update the local best solution for a particle outside of the boundaries derived from the constraints, particles that end up outside will eventually turn back moving towards the local and global best.

Technical documentation for the implementation of the constraints can be found [here](#).

3.5. CD Pipeline

In order to always have the latest documentation updates ready and at disposal of each member of the project, a Continuous Deployment (CD) pipeline was setup to automatically render and publish the Doxygen documentation, without having to build it locally every time. This kind of setup proved to be extremely useful also when in need to reference another person to some particular class member or method.

As noted before, the full documentation is available at <https://etabeta1.github.io/particle-swarm-optimization/>. The full definition of the pipeline is available [here](#).

The pipeline makes use of an Ubuntu container in which Doxygen gets installed and called that also executes the `actions/checkout@v3`, `actions/upload-pages-artifact@v3` and `actions/deploy-pages@v4` actions that are provided by GitHub. Figure 2 shows the various steps followed automatically by the action.

The `Doxyfile` provided to Doxygen is the de-

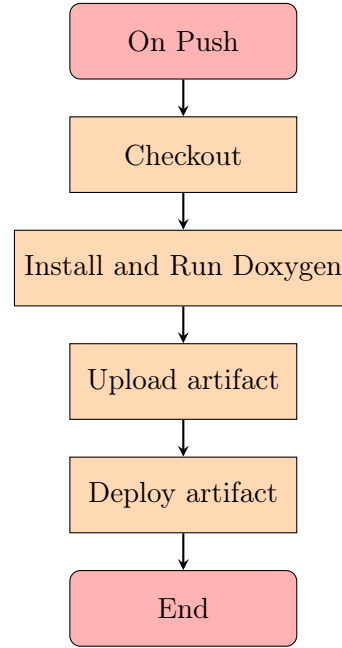


Figure 2: Steps followed by the GitHub Actions workflow

fault configuration (i.e. the one obtained by running `doxygen -g`) on which some minor modifications have been applied in order to, for example, enable Mathjax and point Doxygen to the correct folder containing the sources.

The commit hash and timestamp the documentation refers to, is always shown on the top left corner of the documentation website.

3.6. Unit tests

A Unit testing system was created in order to automatize testing of the most basic components of our project.

This system actually is composed by a single test target `Makefile` (located the same `Makefile` that also manages compilation of examples, docs, etc.).

In order to standardize how tests are written, a `TestManager` class was introduced. The role of this class is to store, run and record the results of each `Test`. One `Test` is a struct containing an `std::string` (to identify and describe the test) and a lambda function returning either `PASS` or `ERR` (both are defined in a specifically made enum) depending on whether the test has passed or not.

Test sources are available [here](#). In the same

folder, the file `tests.hpp` contains the source for the `TestManager` class.

When `make test` is invoked, all the sources in the `tests` folder are sequentially compiled and run.

Currently, we only test the *basic* building blocks of the library (i.e. `chaosmap.hpp`, `constraint.hpp`, `function.hpp` and `point.hpp`) without testing the more complex stuff (like `swarm.hpp` and `particle.hpp`): due to their nondeterministic nature, we thought that, instead of writing specific code to get around nondeterminism, injecting predefined values, it was best to just create a series of reproducible examples and benchmarks that can also show and plot how different configurations of the different algorithms converge.

The project has been submitted with 100% of the tests passing.

4. Benchmarks and comparisons

Benchmarks have been performed on a CPU node of the DMAT cluster. We tested our algorithms with different sized problems on a variable number of processors.

Each CPU node features a 112 cores Intel Xeon Gold 6238R (of whose cores we can only use up to 28) with 512GiB of RAM (of which we can use only around 503).

The benchmark has been launched using a PBS script to reserve the computational resources needed by the benchmarks.

Here we illustrate the main results we observed.

All the sources run to perform the benchmarks can be found under the `app` folder in the project. Benchmark results are saved in JSON and TXT formats in `benchmark` and plotted using a custom Python script.

4.1. Speed benchmarks

4.1.1 CHOPSO benchmarks

Figures 3 and 4 depict the time, in linear and log-log scales, respectively, it takes for our CHOPSO implementation to complete a fixed number of iterations (8192) with different numbers of processors (from 1 to 28) and different

number of particles ($10 \cdot 10^3$, $40 \cdot 10^3$ $16 \cdot 10^4$ per type).

The linear scale graph indicates that execution time is inversely proportional to the number of cores, following the ideal scaling law $T(P) \approx T_1/P$. This behavior is validated by the log-log plot, where the data points align along a linear trend with a negative slope, characteristic of a power-law decay.

As we could not use more than 28 processors, we could not further investigate this behavior beyond this limit but the performance degradation observed as the thread count approaches this limit is a classical phenomenon in High Performance Computing, primarily driven by memory bandwidth saturation and synchronization overhead.

From an architectural standpoint, the Intel Xeon Gold 6238R features exactly 28 physical cores per socket. As the OpenMP thread count nears this physical limit, the internal mesh interconnect and the memory controllers become heavily saturated. Because particle updates require streaming large arrays of coordinates and velocities, the algorithm becomes memory-bound. When 24 to 28 threads simultaneously demand data from the main memory, they hit the memory bandwidth ceiling (the von Neumann bottleneck), causing cache eviction rates and memory access latencies to spike drastically. [1]

Mathematically, this deviation from linear scaling is modeled by an extended version of Amdahl's Law, where the speedup $S(p)$ incorporates a thread-dependent penalty function $\kappa(p)$:

$$S(p) = \frac{1}{(1 - f) + \frac{f}{p} + \kappa(p)}$$

In our CHOPSO implementation, $\kappa(p)$ is strictly dominated by the OpenMP reduction clause used to aggregate the global best evaluated point. As the number of threads p increases, the cost of the hardware-level reduction tree, which scales as $O(\log p)$, along with the cache coherency traffic (MESI protocol) required to synchronize the shared state across all 28 cores, begins to overshadow the computational time saved by dividing the floating-point workload.

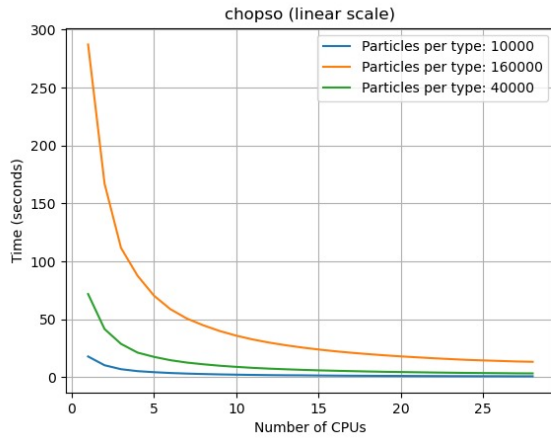


Figure 3: CHOPSO benchmark results, linear scale, no additional constraint, Intel Xeon Gold 6238R.

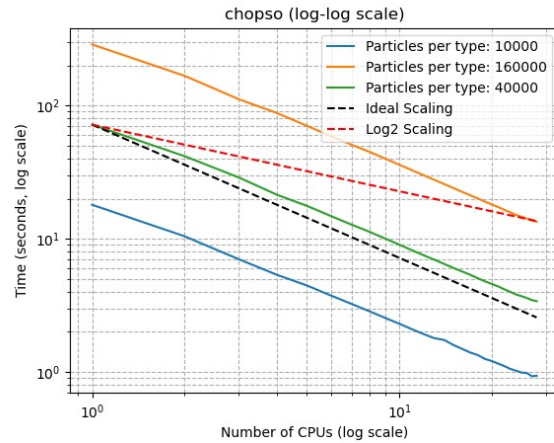


Figure 4: CHOPSO benchmark results, loglog scale, no additional constraint. Intel Xeon Gold 6238R.

Another confirmation of the good scaling capability of our implementation comes from the fact that from the logarithmic plot we can also observe that multiply the amount of particles for a constant a also makes the total time almost multiple of a .

4.1.2 ELPSO benchmarks

The ELPSO implementation, being benchmarked in the same exact way, shows similar behaviors. In particular we observe again a good ability to scale and a quasi-logarithmic decay of the time it takes for the algorithm to complete the computation. Compared to CHOPSO, our ELPSO implementation takes more time per iteration but this is compensated by the fact that it takes a smaller number of iterations to arrive at a good result as we can see in Figure 22 and Figure 60 (more visual result in the correlated graphics).

In addition, we observe two differences. First, the timing curves are much less smoother, compared with the CHOPSO ones and then in the logarithmic scaling plot the performance curves exhibit notable fluctuations, often referred to as jitter, and begin to deviate from the ideal scaling line at higher core counts (when $p > 16$). This interference is particularly pronounced for the smallest workload ($10 \cdot 10^3$ particles).

From a theoretical perspective, the expected parallel execution time $T(p)$ deviates from the ideal $T_{\text{ideal}}(p) = \frac{T_1}{p}$ due to a non-linear overhead

penalty function $\kappa(p, N)$. For a small N and a large p , the workload granularity per thread, defined as $W = \frac{N}{p}$, becomes extremely small. When W drops below a certain computational threshold, the fixed costs dictated by $\kappa(p, N)$ —such as the $O(\log p)$ latency of OpenMP implicit barriers at the end of parallel regions—begin to dominate the actual floating-point payload.

Our hypothesis is that the jitter specifically arises because thread scheduling and barrier wait times are non-deterministic and heavily influenced by the operating system's context switching in order to compute the genetic mutations. This is caused by the fact that within the ELPSO implementation for each particle, each possible mutations evaluated in parallel. This introduces two primary hardware-level bottlenecks that manifest as the observed interferences: NUMA Latency and Memory Contention.

Finally, we observe that a complete simulation run of the algorithm is slower than a CHOPSO one independently from the numbers of cores used.

The possible reason behind it, is that Mutation evaluation takes more time to perform a mutation, rather than performing a one cycle of iteration of CHOPSO algorithm. Because the overhead of mutation processes is fixed, and it doesn't change with respect to the number of cores.

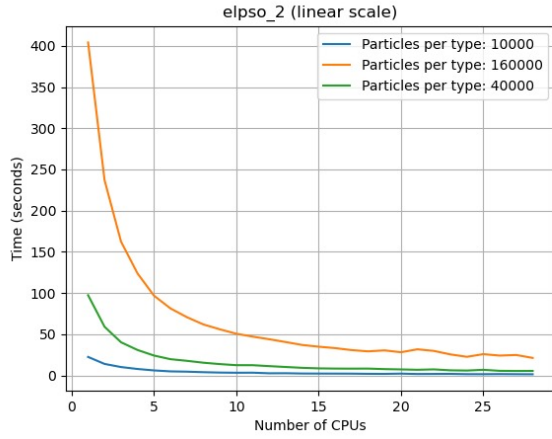


Figure 5: ELPSO benchmark results, linear scale, no additional constraints. Intel Xeon Gold 6238R.

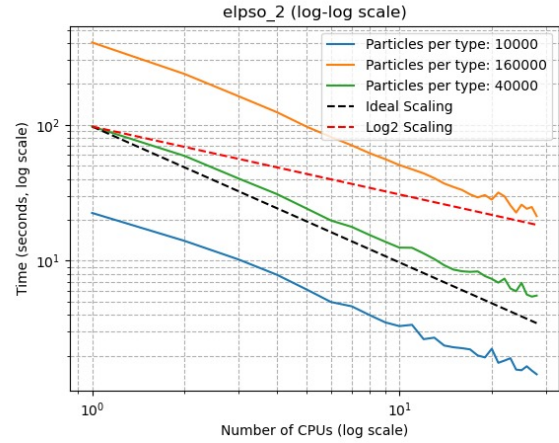


Figure 6: ELPSO benchmark results, log-log scale, no additional constraints. Intel Xeon Gold 6238R.

4.1.3 Simulated Annealing benchmarks

When comparing the benchmark results, obtained with the same procedure, of the Simulated Annealing variant (SAopt, 7 8) against the standard CHOPSO algorithm and the Genetic Optimizer, a striking behavioral divergence emerges.

While CHOPSO exhibits highly predictable strong scaling and ELPSO maintains a relatively smooth degradation curve, SAopt is characterized by severe jitter and chaotic performance spikes. This volatility is particularly pronounced in the high-workload scenario ($16 \cdot 10^4$ particles) as the thread count exceeds 15 CPUs.

This divergence stems from the core mechanism of the Simulated Annealing heuristic. CHOPSO deterministically updates a particle's trajectory based on explicitly evaluated gradients or positional history. In contrast, SAopt incorporates the thermodynamic Metropolis-Hastings acceptance criterion. A mathematically worse candidate position is not immediately discarded but is conditionally accepted with a probability P :

$$P = \exp \left(-\frac{f(\mathbf{x}_{\text{candidate}}) - f(\mathbf{x}_i^{(k)})}{T_k} \right)$$

To evaluate this probabilistic condition, the algorithm must generate a uniformly distributed pseudo-random number $r \sim \mathcal{U}(0, 1)$ and verify if $r < P$. At the hardware and architecture level, generating high-entropy pseudo-random

numbers in a highly concurrent OpenMP environment introduces severe bottlenecks that impact scalability.

Although both SAopt and ELPSO rely on pseudo-random number generators (PRNGs), the number of times it is used per iteration differs vastly, explaining why ELPSO scales better.

In our ELPSO implementation, random genetic mutations (such as Gaussian or Uniform perturbations) are applied to the Global Best resulting in a very limited numbers of calls.

Conversely, in SAopt, considering the worst-case scenario, the PRNG is invoked once per particle per iteration. Since the calls happen in an highly parallel situation, if the PRNG relies on a shared global state (e.g., a `std::mt19937` engine, like our implementation does), parallel threads must be synchronized in order to safely extract a sequence and mutate the PRNG's internal state.

This massive need for synchronization effectively serializes the velocity update phase, temporarily reducing the theoretical $O(N/p)$ parallel complexity to $O(N)$ and causing the dramatic execution time spikes observed in 8.

Furthermore, even if the PRNG overhead were mitigated by using strictly thread-local random engines (avoiding shared locks), the intrinsic stochasticity of the SA acceptance rule severely impact the performance, in terms of correct predictions, of the branch prediction algorithm in CPU, introducing many pipeline flushes and stalls.

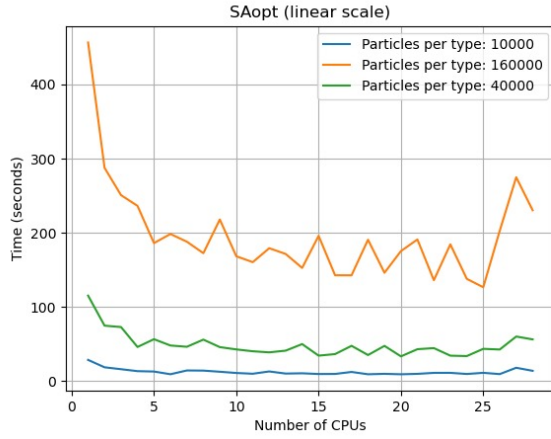


Figure 7: SAopt benchmark results, linear scale, no additional constraint. Intel Xeon Gold 6238R.

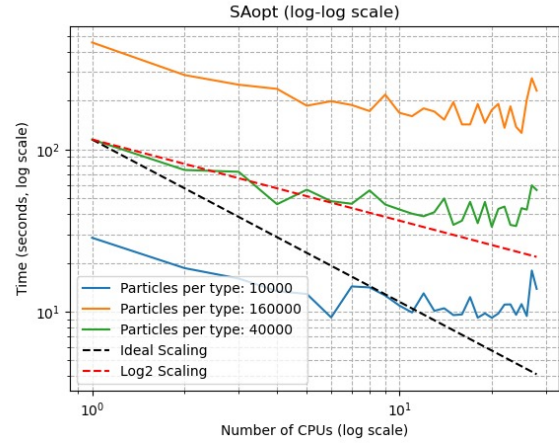


Figure 8: SAopt benchmark results, loglog scale, no additional constraint. Intel Xeon Gold 6238R.

Within a `#pragma omp parallel for block`, like the one we use to update all the particles, static scheduling assigns an equal contiguous chunk of particles to each thread. However, because acceptance depends on random extraction, some threads will rapidly reject candidates and bypass complex memory write operations, while others will probabilistically accept them, triggering L1 cache updates and state modifications. This non-deterministic control flow results in significant load imbalance. Consequently, fast-finishing threads stall at the implicit OpenMP synchronization barrier at the end of the loop (`#pragma omp barrier`), wasting CPU cycles.

When comparing the benchmark results of the single-constraint CHOPSO (94) with the unconstrained CHOPSO, we can indeed observe a slight vertical shift of the execution time curves.

For instance, the baseline execution time for the $16 \cdot 10^4$ particles workload on a single core increases from approximately 280 seconds to roughly 315 seconds. This overall increase was absolutely to be expected.

From a mathematical optimization perspective, restricting the global search space to a feasible region Ω means that for every newly computed candidate position $\mathbf{x}(k+1)$, the algorithm must evaluate at least one constraint inequality $g(\mathbf{x}^{(k+1)}) \leq 0$ before proceeding with any personal or global best updates. From an HPC and computer architecture standpoint, introducing constraints creates a complex computational trade-off. According to the algorithm's

design, the personal best solution is updated only if all constraints are respected, which intentionally avoids computing the objective function for out-of-bounds particles.

While this saves the floating-point operations (FLOPs) associated with the objective function $f(\mathbf{x})$, the net increase in total execution time suggests that the overhead of evaluating the non-linear constraint $g(\mathbf{x})$, combined with the necessary conditional branching (if `is_feasible`), outweighs the saved computational cycles. This may be different when considering objective functions that are much more complex to compute than the ones we used as a test.

At the hardware level, this conditional logic puts pressure on the CPU's branch predictor. When chaotically moving particles frequently cross the feasible boundary, it leads to branch mispredictions, forcing the CPU to flush its instruction pipeline and stall. However, because the evaluation of $g(\mathbf{x})$ is an entirely local, embarrassingly parallel operation that requires no inter-thread synchronization, it actually increases the strictly parallelizable fraction f within Amdahl's formulation.

Consequently, the relative scaling efficiency remains incredibly robust, as evidenced by the curves in the log-log plot 10 which maintain a trajectory tightly parallel to the ideal scaling baseline even at higher core counts.

While for this considerations we only took into account the CHOPSO algorithm, the same can

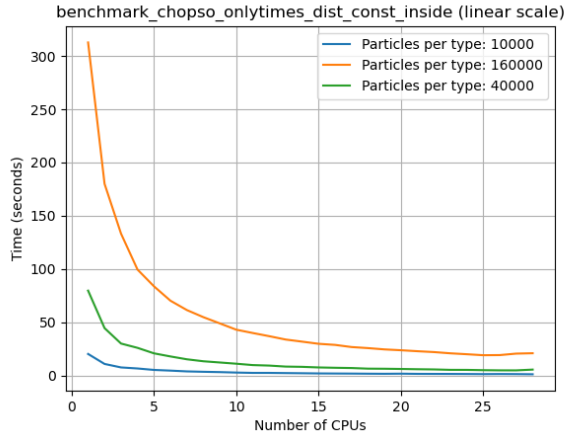


Figure 9: CHOPSO benchmark results, linear scale, one additional constraint. Intel Xeon Gold 6238R.

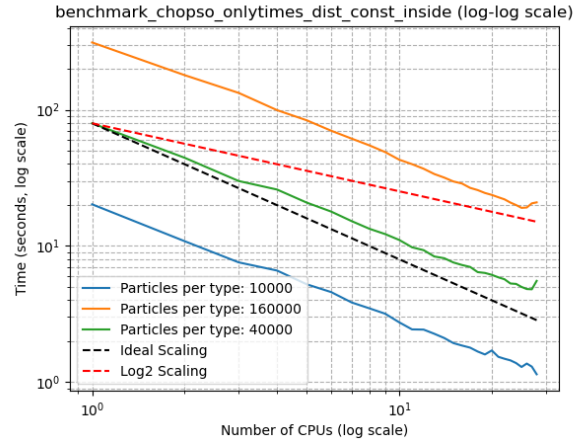


Figure 10: CHOPSO benchmark results, loglog scale, one additional constraint. Intel Xeon Gold 6238R.

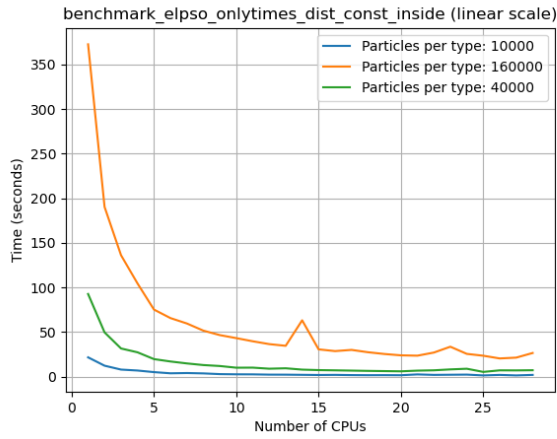


Figure 11: ELPSO benchmark results, linear scale, one additional constraint. Intel Xeon Gold 6238R.

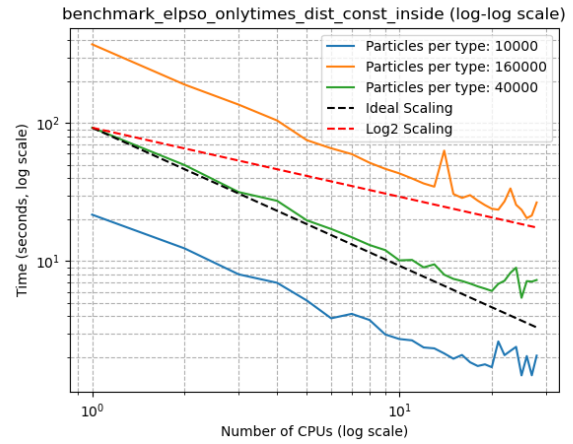


Figure 12: ELPSO benchmark results, loglog scale, one additional constraint. Intel Xeon Gold 6238R.

be said for the ELPSO one except that its timings are subject to more jitter.

Regarding SAopt, it seems that with an additional constraint the algorithm is faster when it has access to more than 15 cores.

The same trend (i.e. an upward shift in the computation time), continues to manifest itself when increasing the number of constraints applied (Figures 15 through 20).

4.2. Convergence tests

We will now go through and compare all the convergence tests we have performed on our implementations. Unless differently noted, all the plots will represent the first 256 iterations of an

8192 iterations simulation. To allow for better comparisons between the different algorithms, we try to keep all the conditions the same (same number of particles, same 10x10 domain, same chaotic map).

Unless differently noted, since all the functions used in the convergence tests, when evaluated at their global optimum, give 0, all the referenced plots in this section may also be read as the error between the approximations at each iteration and the exact solution.

In order to not disrupt readability, all the convergence plots will be placed in the appendix of the present document

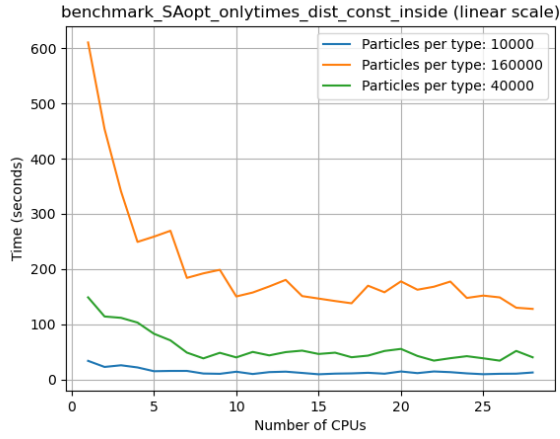


Figure 13: SAopt benchmark results, linear scale, one additional constraint. Intel Xeon Gold 6238R.

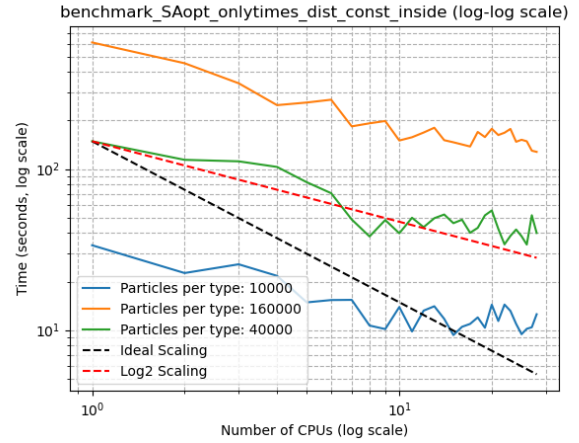


Figure 14: SAopt benchmark results, loglog scale, one additional constraint. Intel Xeon Gold 6238R.

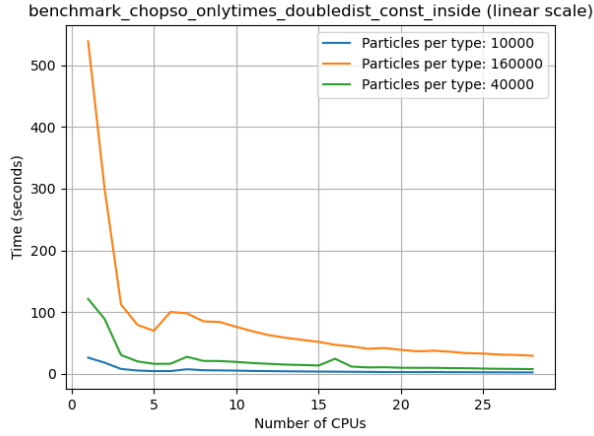


Figure 15: CHOPSO benchmark results, linear scale, 2 additional constraints. Intel Xeon Gold 6238R.

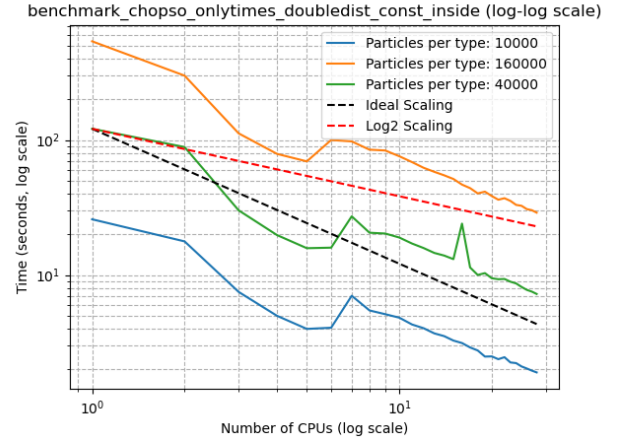


Figure 16: CHOPSO benchmark results, log-log scale, 2 additional constraints. Intel Xeon Gold 6238R.

4.2.1 CHOPSO convergence

In addition to the aforementioned conditions, the CHOPSO convergence test has been performed with 32 particles per type, $c_1 = c_2 = 2.5$ and a Chebyshev chaotic map.

Figures 27 through 32 show how our CHOPSO implementation converges when approximating the global minimum of various examples of objective functions. We observe that the CHOPSO algorithm is very efficient: we observe that for **easy** functions, like the Ellipsoid (Figure 30, the Quintic (Figure 31) and the Sphere (Figure 32, convergence to the optimum is almost immediate, taking in some cases less than 10 iterations.

Knowing that the Ackley function, the Alpine1

function and the Dropwave function are more difficult to optimize (they have a lot of local minima in which particles can get stuck), we should not be surprised to observe behaviors such as the ones depicted in Figures 27, 28 and 29.

In those plots we can see a sudden initial improvement, given by the huge exploratory power of the chaotic particles, followed by a phase of fine improvements given by the normally moving particles carefully exploring the surroundings of the "good enough" position found by their chaotic colleagues.

In Figure 27 we observe an interesting phenomenon where the particles, after at least one of them traveling really close to the global op-

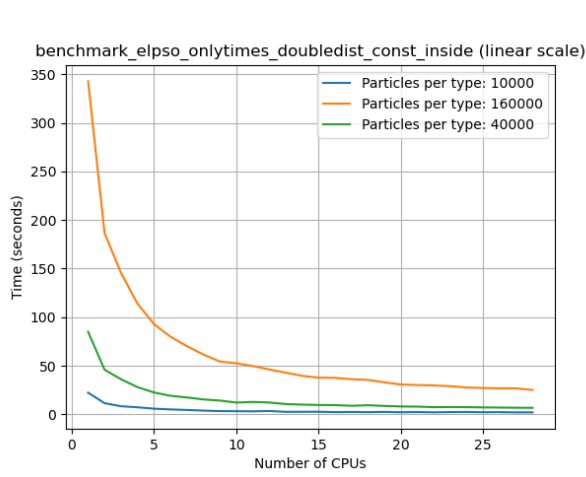


Figure 17: ELPSO benchmark results, linear scale, 2 additional constraints. Intel Xeon Gold 6238R.

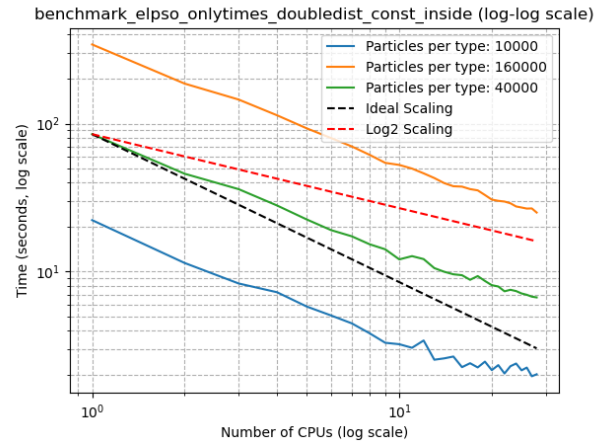


Figure 18: ELPSO benchmark results, log-log scale, 2 additional constraints. Intel Xeon Gold 6238R.

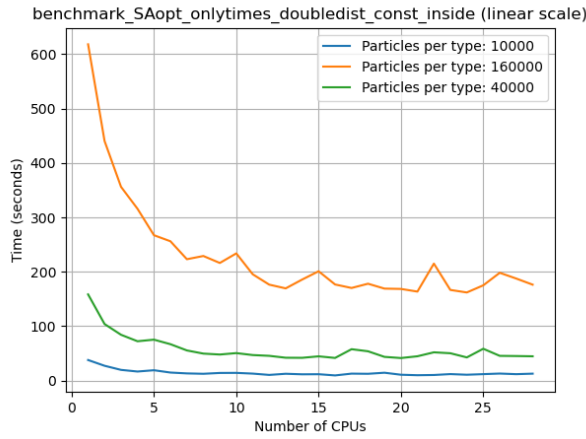


Figure 19: SAopt benchmark results, linear scale, 2 additional constraints. Intel Xeon Gold 6238R.

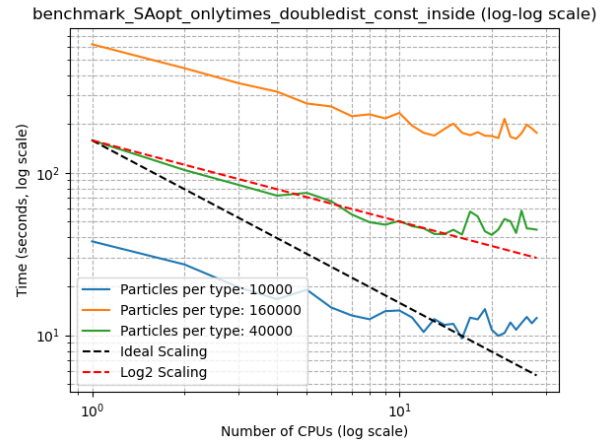


Figure 20: SAopt benchmark results, loglog scale, 2 additional constraints. Intel Xeon Gold 6238R.

timum, get stuck into another local minimum without. Between iteration 150 and 200 it is possible to observe the particles trying to escape, without much success. In such a case, an higher social coefficient (or a lower cognitive one) may have helped in preventing this.

Even with this minor incident, our CHOPSO implementation was able to provide a good approximation of the solution of the problem nevertheless.

Within Figures 22 through 26 we can see that for a difficult function such as the Ackley one, the Singer chaotic map seems to provide the best results. We also observe that the Ackley function is particularly prone to making the particle

get stuck in local minima.

4.2.2 SAOptimizer convergence

Figures 44 through 48 illustrate how the Simulated Annealing (SA) implementation converges toward the global minimum for various objective functions (Ackley, Alpine1, Dropwave, Ellipsoid, Quintic), tested in combination with different chaosmaps (iterative, logistic, sine, singer, sinusoidal).

In the presented plots, we can observe the efficiency of the algorithm on functions of varying complexity. For relatively easy and convex (or quasi-convex) functions such as the Ellipsoid (with a singer chaosmap, Figure 47) and

the Quintic (with a sinusoidal chaosmap, Figure 42), convergence to the optimum is extremely rapid. The initial fluctuations dampen quickly, and the system settles on the global minimum in less than 50-100 iterations.

Knowing that functions like Ackley, Alpine1, and Dropwave are more difficult to optimize due to the high density of local minima where the search can get trapped, the behavior shown in their respective graphs (Figures 38, 39, and 40) is perfectly in line with SA theory.

In these plots, a division into two distinct phases dictated by the temperature trend T is clearly visible:

- **Exploration Phase (High Temperature):** In the initial iterations (typically from 0 to 50), the orange line (*Minimum of local best*, representing the evaluations of the currently explored states) exhibits extreme fluctuations. Mathematically, a high value of T ensures that $P(\Delta E, T) \approx 1$, allowing the algorithm to frequently accept "worsening" moves to escape local minima.
- **Exploitation Phase (Low Temperature):** As the temperature decreases, the fluctuations drastically reduce. The algorithm begins to behave like a simple gradient descent (or *hill climbing*), refining the solution within the optimal basin of attraction it has landed in.

In the case of the Dropwave function (with a sine chaosmap, Figure 46), a very interesting phenomenon is observed: the Global best (blue line) remains stuck on a plateau (roughly between iterations 10 and 30). This indicates that the best solution found up to that point is trapped in a local minimum. However, by observing the orange line above it, we see the system continuing to explore worse states. It is precisely thanks to these background oscillations that, around iteration 30, the algorithm manages to overcome the barrier of the local minimum and "fall" into a deeper valley, instantly updating the Global best to a value close to zero.

4.2.3 GENETICOptimizer convergence

Figures 60 through 64 show how our ELPSO implementation converges when approximating the global minimum of various objective functions, combined with different mapping strategies.

We can observe that the ELPSO algorithm is efficient for easy and structurally simple functions. For instance, in the Ellipsoid function (Figure 63) and the Quintic function (Figure 64), convergence to the global optimum is nearly instantaneous. Remarkably, for the Quintic function, the objective value drops from almost 3500 to essentially zero in less than 5 iterations. This demonstrates the algorithm's aggressive exploitation capabilities when the gradient of the basin of attraction strictly leads to the global minimum.

Conversely, for more difficult multimodal functions, such as Ackley (Figure 60), Alpine1 (Figure 61), and Dropwave (Figure 62), the search space is densely populated with local minima. In these plots, we observe a distinct step-like behavior in the Global best (blue line) alongside sharp, high-frequency oscillations in the Minimum of local best (orange line).

These high oscillations during the initial iterations (roughly between iteration 0 and 40) visually represent the high kinetic energy of the swarm. Driven by the inertia weight w and the stochastic components $r1$, $r2$, the particles broadly sample the domain. When the Global best line plateaus (as seen between iterations 10 and 20 in the Alpine1 plot), the swarm is temporarily trapped in a local minimum. However, the continuous fluctuation of the local best indicates that the particles are persistently exploring adjacent regions. Eventually, a particle's updated position vector $x_i(t+1)$ breaches the boundaries of the local basin, finding a lower fitness value and pulling the rest of the swarm toward the new, deeper valley (characterized by the sudden drops in the blue line).

This rapid stabilization is highly desirable. The fast reduction in the orange line's variance implies that the swarm quickly reaches consensus, minimizing the need for prolonged, computationally expensive objective function evaluations across thousands of iterations, thus maximizing hardware utilization and thread efficiency during parallel fitness evaluations.

5. Conclusions

During this project we implemented and compared three different algorithms used to solve continuous multidimensional optimization problems over an objective function that does not

need to have an existing gradient. Our implementations support the imposition of any kind of nonlinear constraint.

The algorithms were parallelized through the use of OpenMP and benchmarked on a capable machine.

Here we describe the key insights we discovered during the comparison of the algorithms.

CHOPSO is the algorithm with the fastest iteration time but also the one that requires the most iterations to converge.

ELPSO takes more time per iteration than CHOPSO but a single iteration is more effective, therefore it needs less iterations to converge compared to CHOPSO.

With both those two algorithms, the time needed to perform a full run grows proportionally to the numbers of constraints applied.

SAoptimizer's effectiveness and efficiency increase proportionally to the number of constraints but, without any constraints, it's slower compared to CHOPSO and ELPSO.

Due to the high numbers of calls to the PRNG this implementation does not benefit much from an increase in the number of processors, therefore it is a good choice when trying to solve a really constrained problem.

5.1. Future work

This project opened for the possibility of many future improvements in many directions.

One of those directions points towards an improvement in the efficiency and in the parallelization of the algorithms, possibly through the usage of heterogeneous systems including, for example, GPUs, NPUs, FPGAs, TPUs or any other suitable hardware accelerator in general.

Another direction of improvement may be to simply implement more algorithms, maybe targeting different kind of optimization problems like multi-objective and integer optimization in order to make this library more useful in general.

6. Acknowledgements

We would like to thank Paolo Salvi and Marco Melzi, who took part to the initial Hands-On.

A good amount of code written by them is still present in this project. Also many thanks to the DMAT for providing access to their computing power resources.

References

- [1] John Backus. Can programming be liberated from the von neumann style? a functional style and its algebra of programs. *Communications of the ACM*, page 613–641, 08 1978.
- [2] Ahmed Gad. Particle swarm optimization algorithm and its applications: A systematic review. *Archives of Computational Methods in Engineering*, 29:2531–2561, 04 2022.
- [3] Hesham Ahmed Hefny and Shahira Shaaban Azab. Chaotic particle swarm optimization. In *2010 The 7th International Conference on Informatics and Systems (INFOS)*, pages 1–8. IEEE, 2010.
- [4] Fatma Helmy Ismail, Essam H. Houssein, and Aboul Ella Hassanien. Chaotic bird swarm optimization algorithm. In Aboul Ella Hassanien, Mohamed F. Tolba, Khaled Shaalan, and Ahmad Taher Azar, editors, *Proceedings of the International Conference on Advanced Intelligent Systems and Informatics 2018*, pages 294–303, Cham, 2019. Springer International Publishing.
- [5] A. Rezaee Jordehi. Enhanced leader pso (elpso): A new pso variant for solving global optimisation problems. *Applied Soft Computing*, 26:401–417, 2015.

7. Appendix

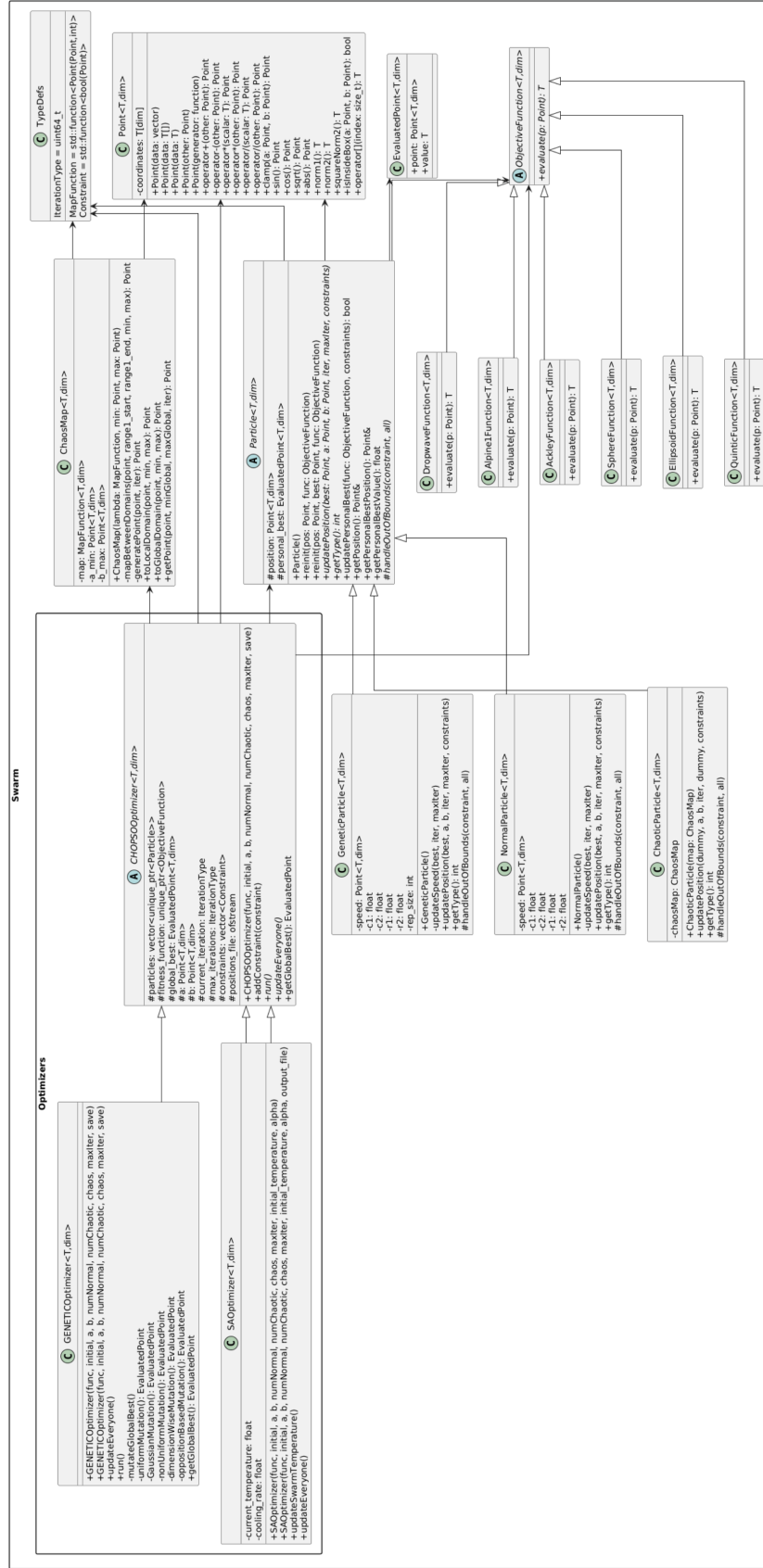


Figure 21: ClassDiagram for the entire project

Chopso convergence plots

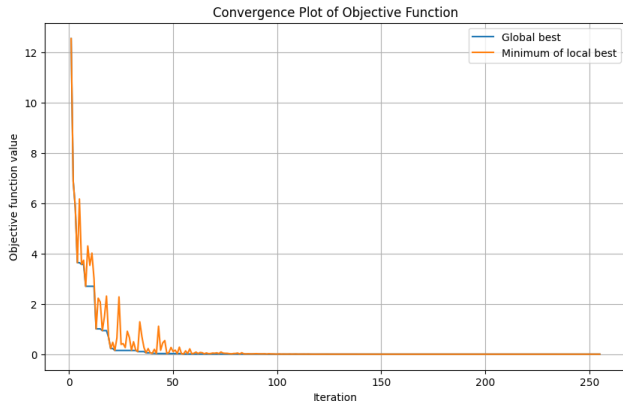


Figure 22: Convergence plot of the Ackley function. Iterative map, 32 particles per type.

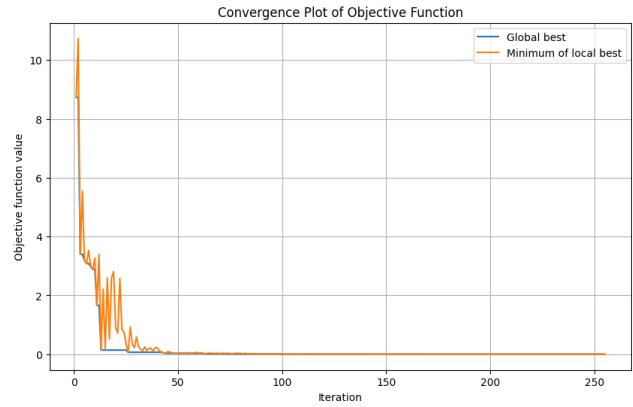


Figure 23: Convergence plot of the Ackley function. Logistic map, 32 particles per type.

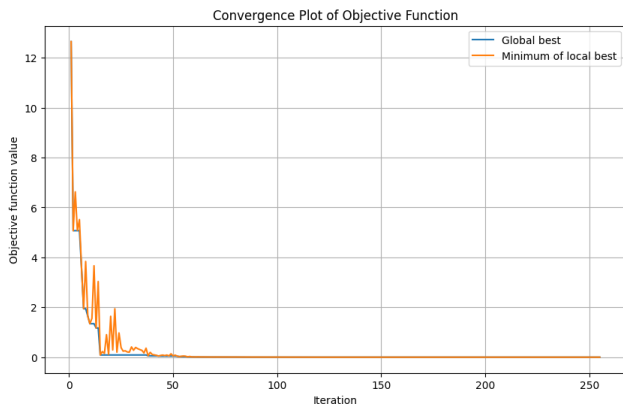


Figure 24: Convergence plot of the Ackley function. Sine map, 32 particles per type.

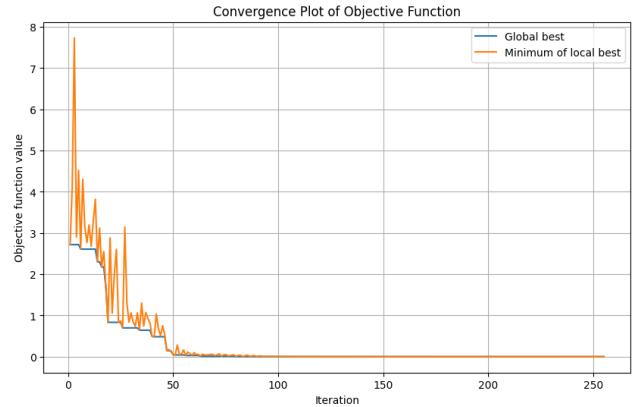


Figure 25: Convergence plot of the Ackley function. Singer map, 32 particles per type.

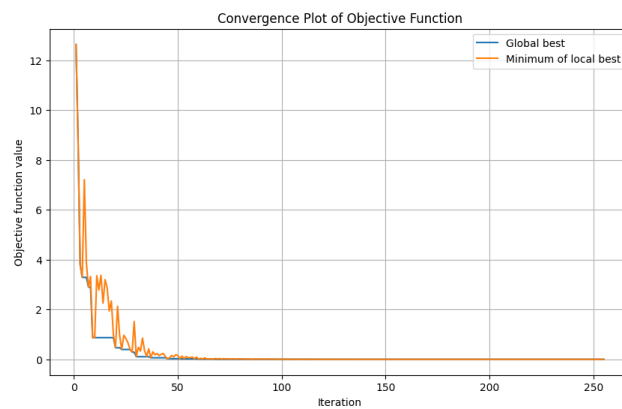


Figure 26: Convergence plot of the Ackley function. Sinusoidal map, 32 particles per type.

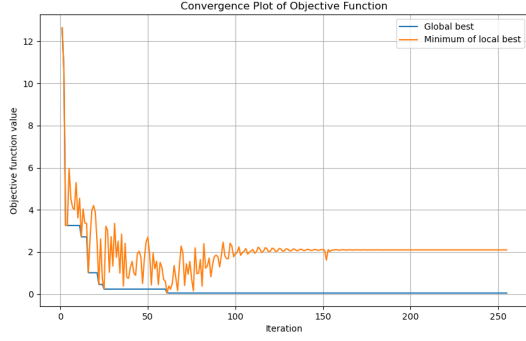


Figure 27: Convergence plot of the Ackley function. Chebyshev Chaotic map, 32 particles per type.

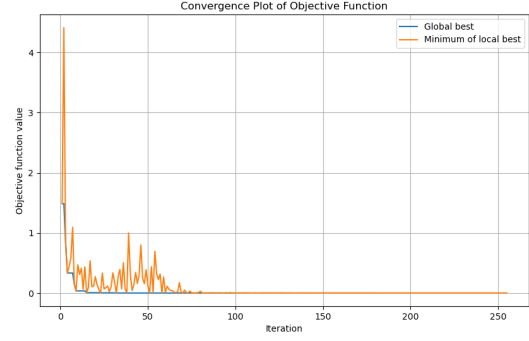


Figure 28: Convergence plot of the Alpine1 function. Chebyshev Chaotic map, 32 particles per type.

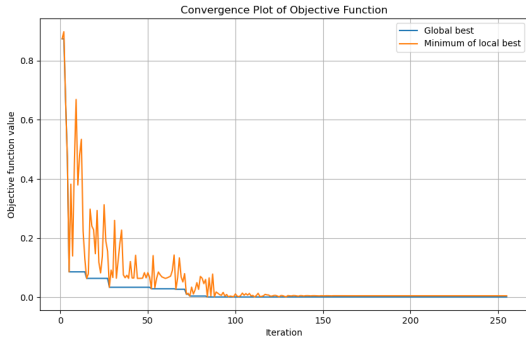


Figure 29: Convergence plot of the Dropwave function. Chebyshev Chaotic map, 32 particles per type.

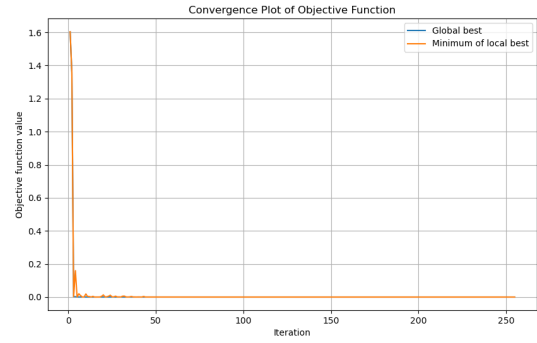


Figure 30: Convergence plot of the Ellipsoid function. Chebyshev Chaotic map, 32 particles per type.

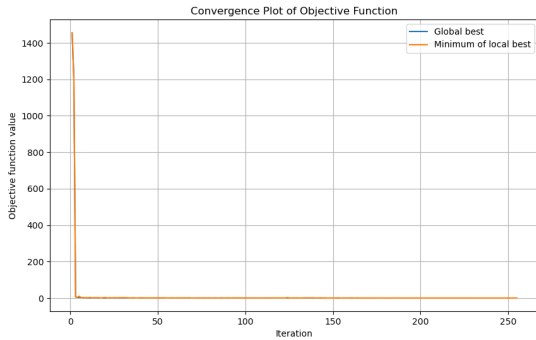


Figure 31: Convergence plot of the Quintic function. Chebyshev Chaotic map, 32 particles per type.

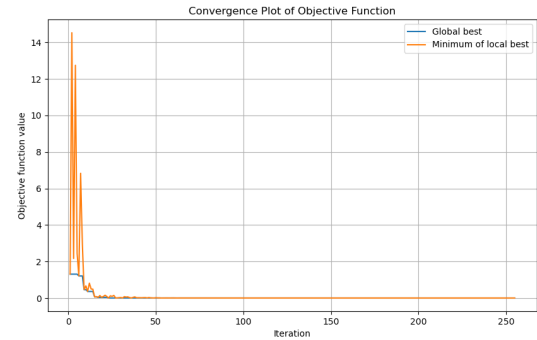


Figure 32: Convergence plot of the Sphere function. Chebyshev Chaotic map, 32 particles per type.

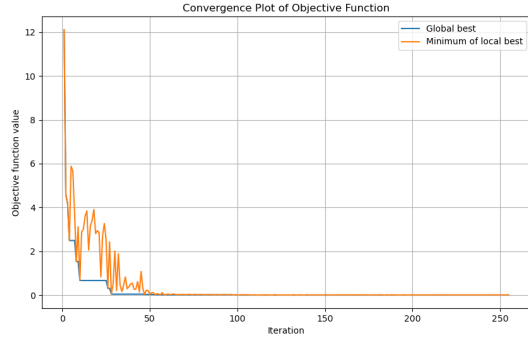


Figure 33: Convergence plot of the Ackley function. Iterative Chaotic map, 32 particles per type.

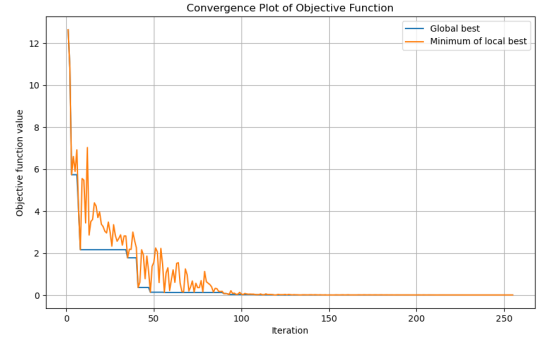


Figure 34: Convergence plot of the Alpine1 function. Logistic Chaotic map, 32 particles per type.

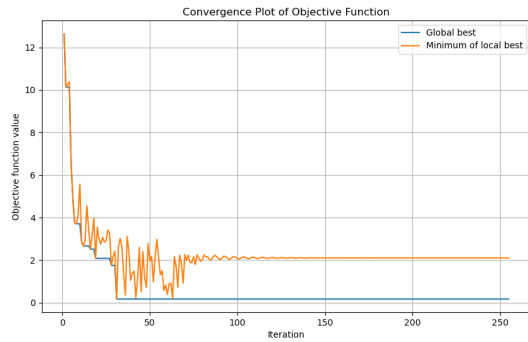


Figure 35: Convergence plot of the Dropwave function. Sine Chaotic map, 32 particles per type.

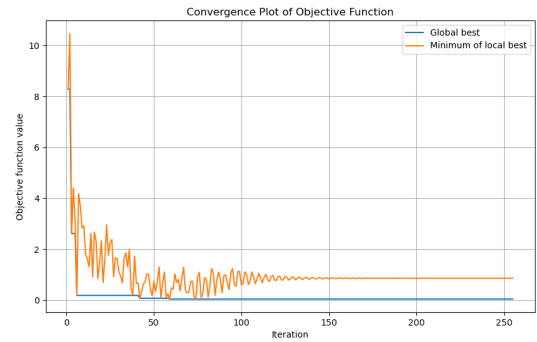


Figure 36: Convergence plot of the Ellipsoid function. Singer Chaotic map, 32 particles per type.

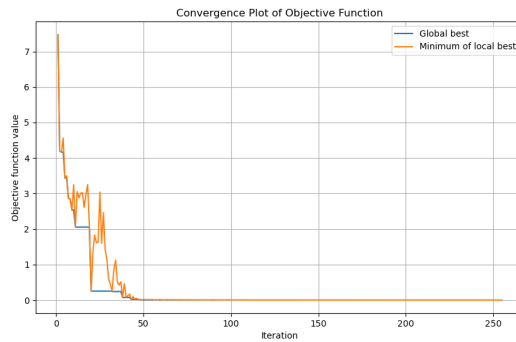


Figure 37: Convergence plot of the Quintic function. Sinusoidal Chaotic map, 32 particles per type.

Simulated Annealing convergence plots

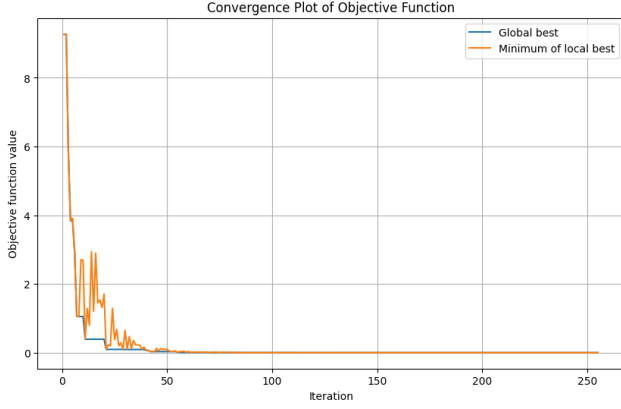


Figure 38: Convergence plot of the Ackley function. Chebyshev Chaotic map, 32 particles per type.

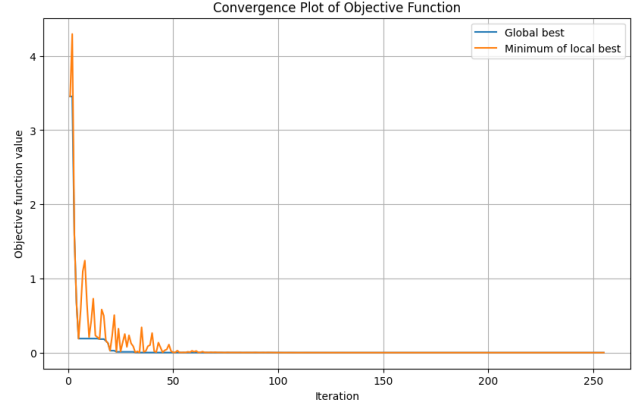


Figure 39: Convergence plot of the Alpine1 function. Chebyshev Chaotic map, 32 particles per type.

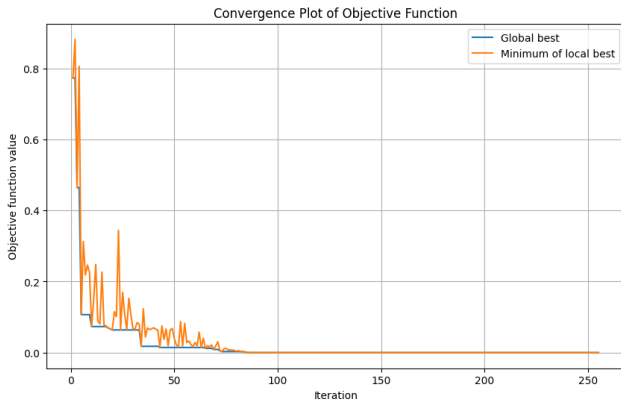


Figure 40: Convergence plot of the Dropwave function. Chebyshev Chaotic map, 32 particles per type.

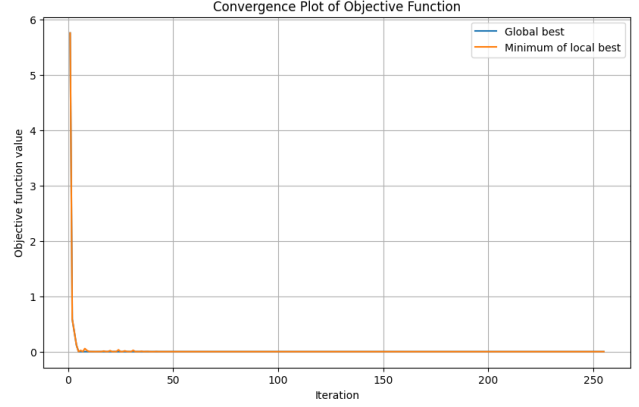


Figure 41: Convergence plot of the Ellipsoid function. Chebyshev Chaotic map, 32 particles per type.

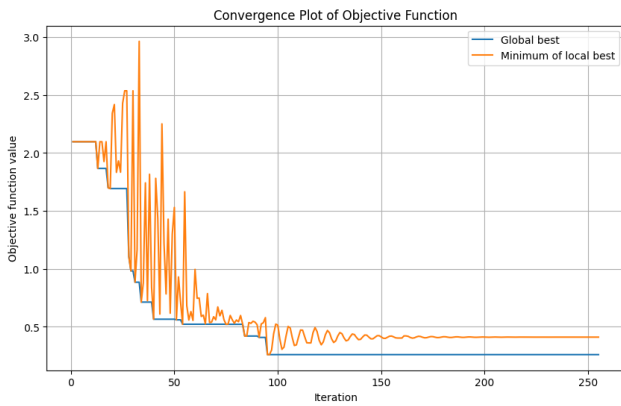


Figure 42: Convergence plot of the Quintic function. Chebyshev Chaotic map, 32 particles per type.

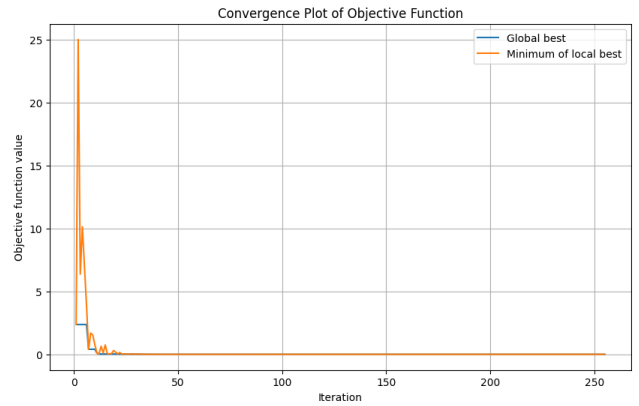


Figure 43: Convergence plot of the Sphere function. Chebyshev Chaotic map, 32 particles per type.

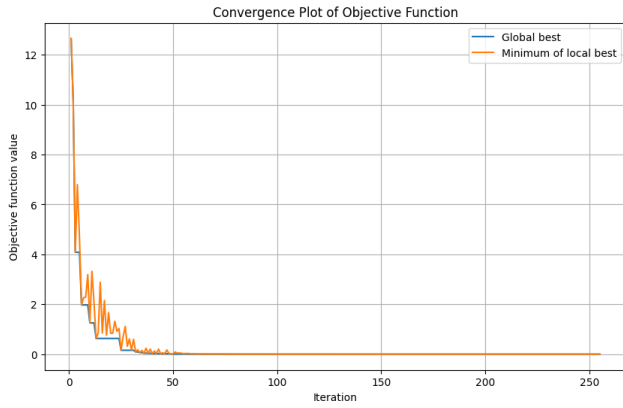


Figure 44: Convergence plot of the Ackley function. Iterative Chaotic map, 32 particles per type.

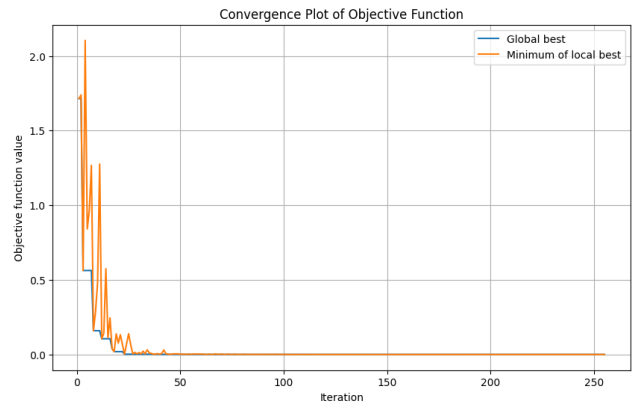


Figure 45: Convergence plot of the Alpine1 function. Logistic Chaotic map, 32 particles per type.

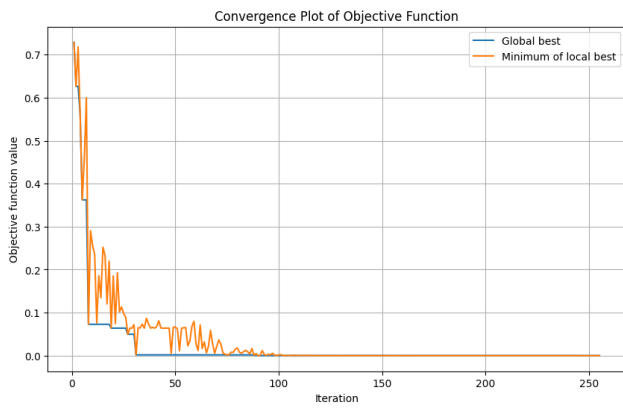


Figure 46: Convergence plot of the Dropwave function. Sine Chaotic map, 32 particles per type.

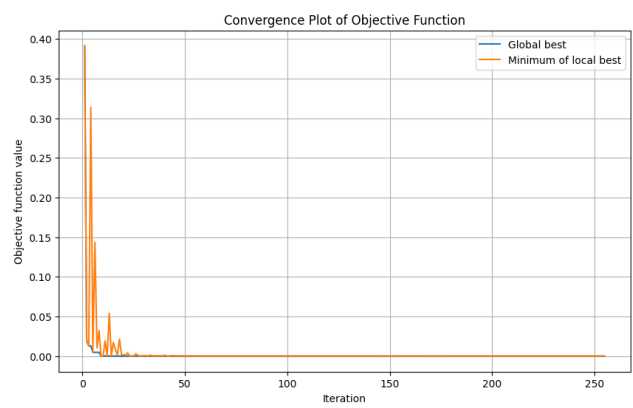


Figure 47: Convergence plot of the Ellipsoid function. Singer Chaotic map, 32 particles per type.

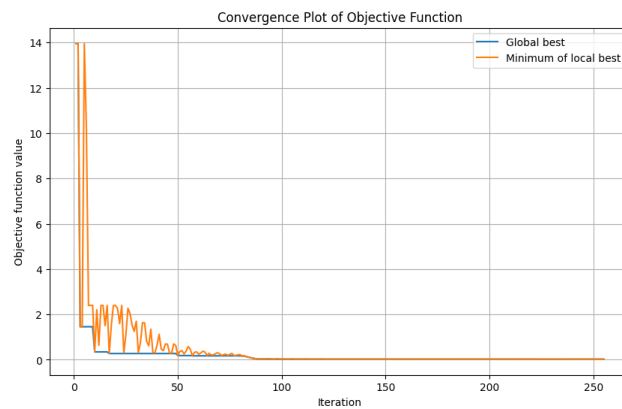


Figure 48: Convergence plot of the Quintic function. Sinusoidal Chaotic map, 32 particles per type.

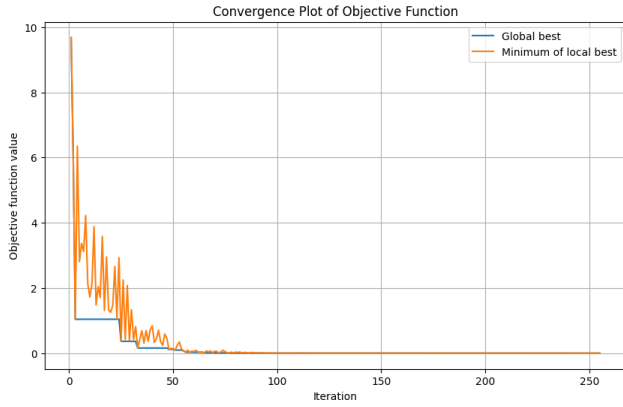


Figure 49: Convergence plot of the Ackley function. Iterative map, 32 particles per type.

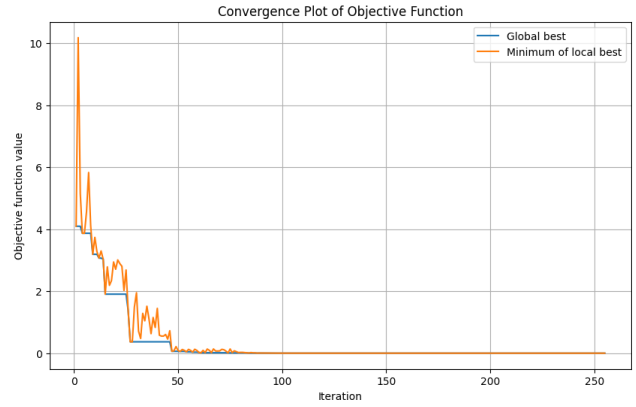


Figure 50: Convergence plot of the Ackley function. Logistic map, 32 particles per type.

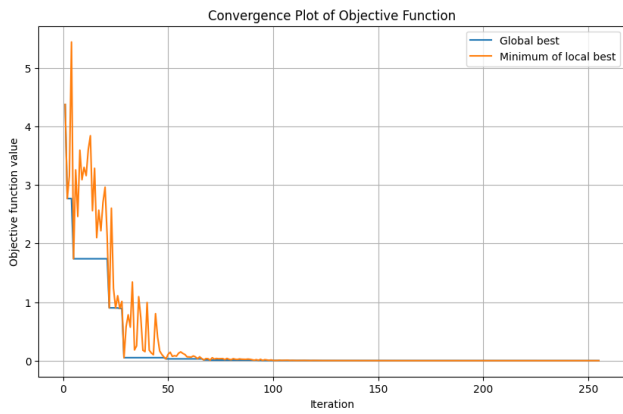


Figure 51: Convergence plot of the Ackley function. Sine map, 32 particles per type.

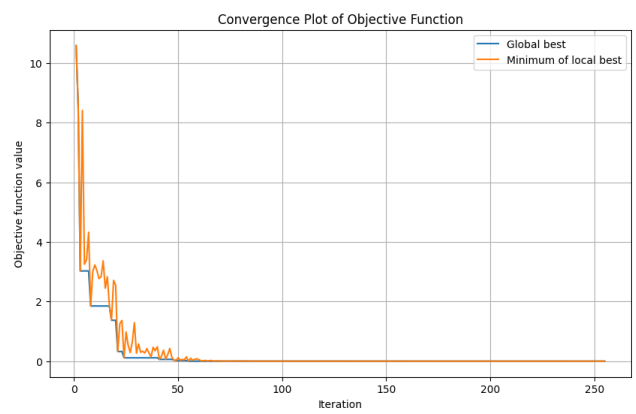


Figure 52: Convergence plot of the Ackley function. Singer map, 32 particles per type.

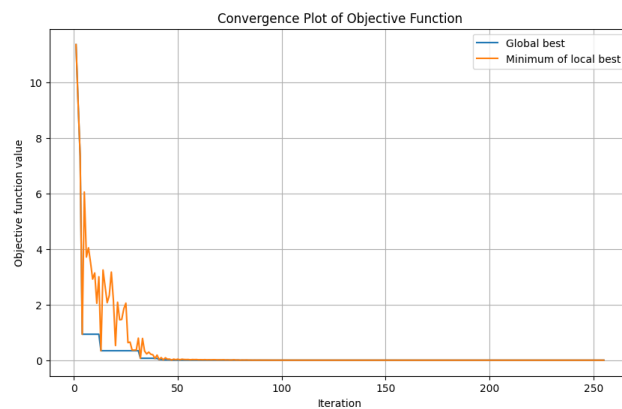


Figure 53: Convergence plot of the Ackley function. Sinusoidal map, 32 particles per type.

ELPSO convergence plots

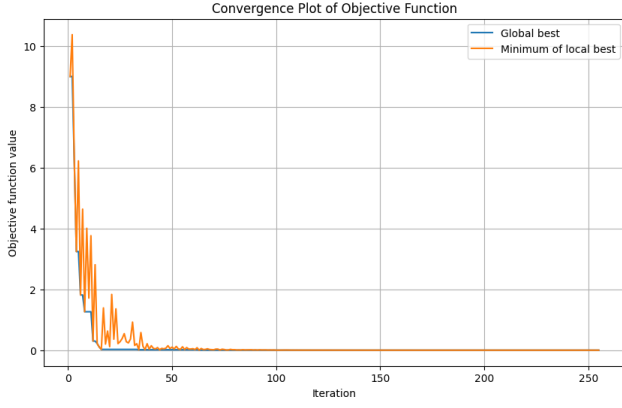


Figure 54: Convergence plot of the Ackley function. Chebyshev Chaotic map, 32 particles per type.

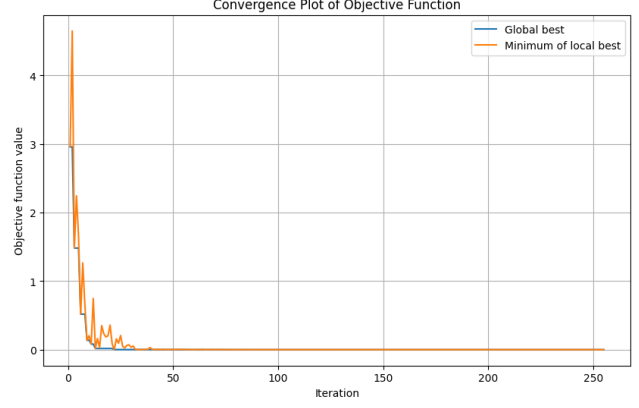


Figure 55: Convergence plot of the Alpine1 function. Chebyshev Chaotic map, 32 particles per type.

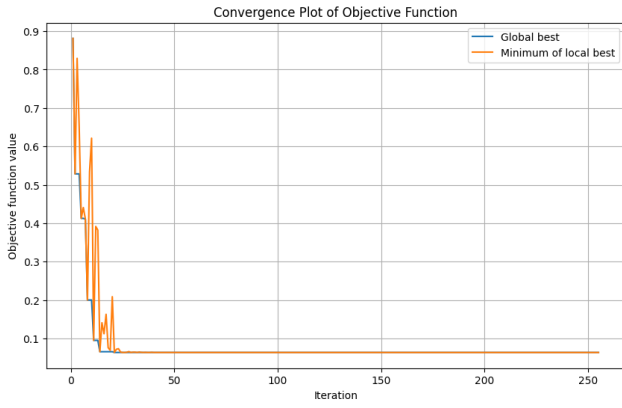


Figure 56: Convergence plot of the Dropwave function. Chebyshev Chaotic map, 32 particles per type.

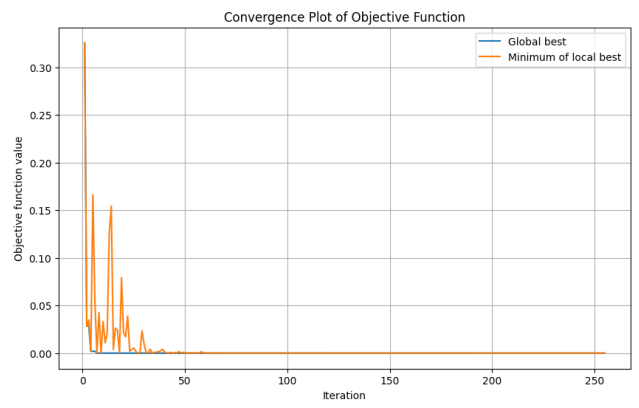


Figure 57: Convergence plot of the Ellipsoid function. Chebyshev Chaotic map, 32 particles per type.

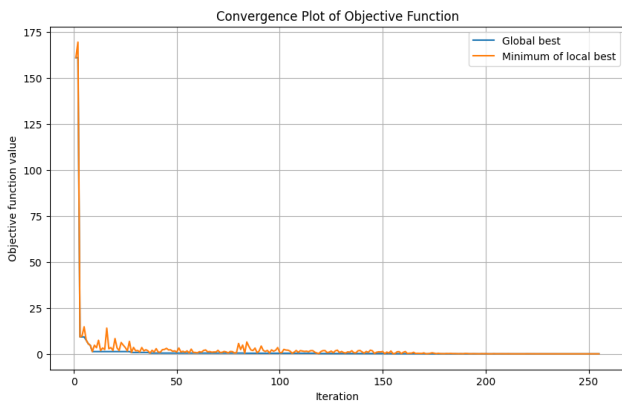


Figure 58: Convergence plot of the Quintic function. Chebyshev Chaotic map, 32 particles per type.

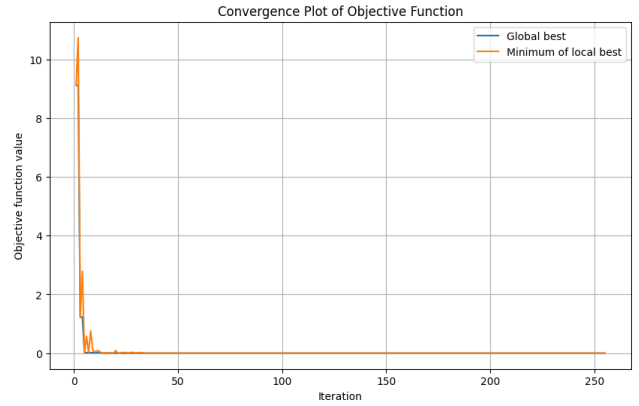


Figure 59: Convergence plot of the Sphere function. Chebyshev Chaotic map, 32 particles per type.

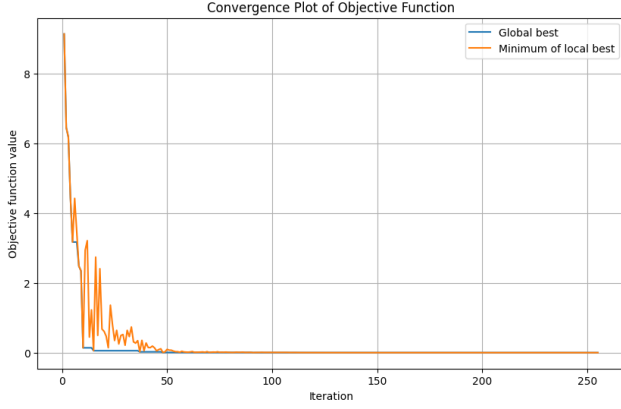


Figure 60: Convergence plot of the Ackley function. Iterative Chaotic map, 32 particles per type.

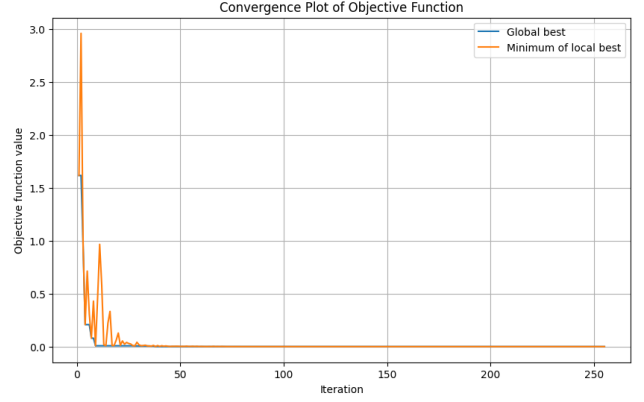


Figure 61: Convergence plot of the Alpine1 function. Logistic Chaotic map, 32 particles per type.

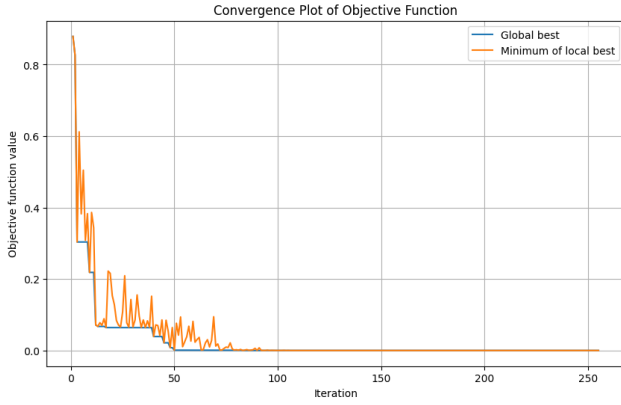


Figure 62: Convergence plot of the Dropwave function. Sine Chaotic map, 32 particles per type.

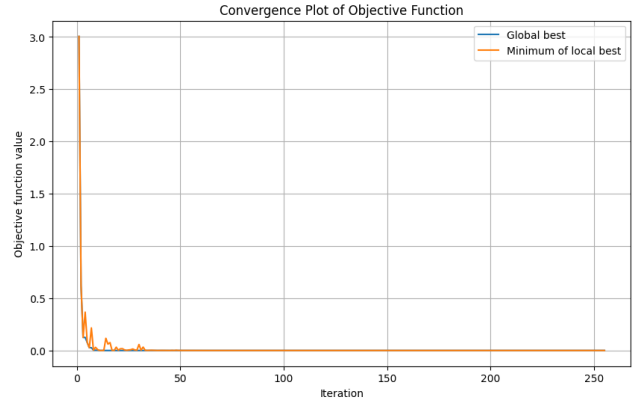


Figure 63: Convergence plot of the Ellipsoid function. Singer Chaotic map, 32 particles per type.

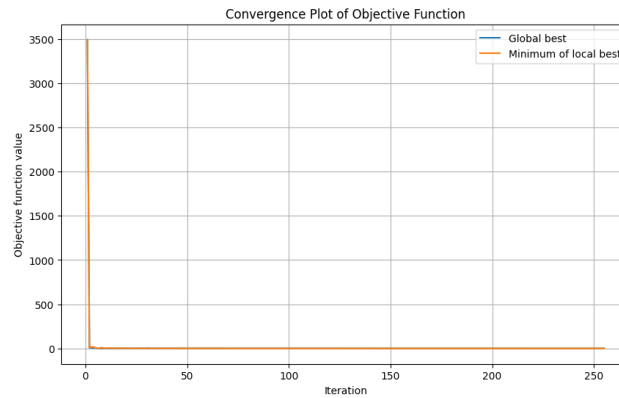


Figure 64: Convergence plot of the Quintic function. Sinusoidal Chaotic map, 32 particles per type.

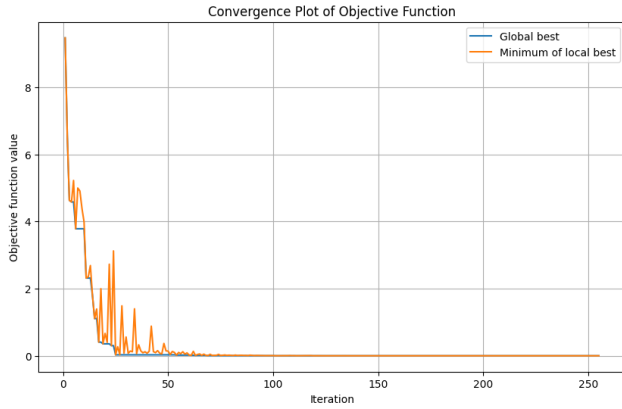


Figure 65: Convergence plot of the Ackley function. Iterative map, 32 particles per type.

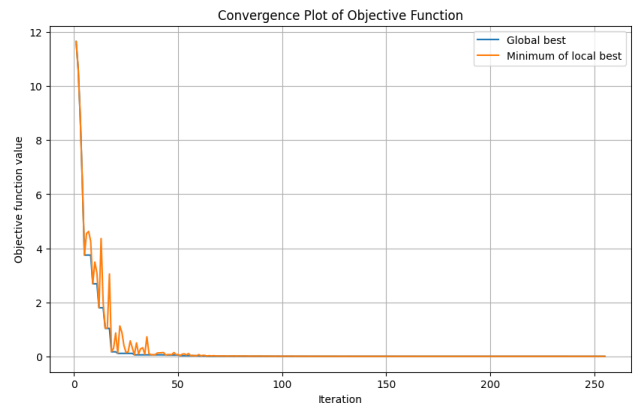


Figure 66: Convergence plot of the Ackley function. Logistic map, 32 particles per type.

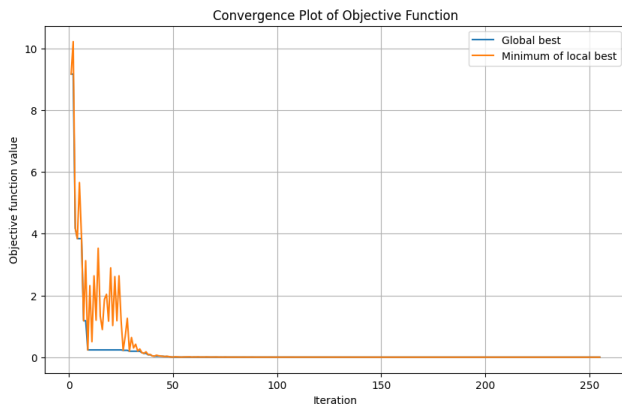


Figure 67: Convergence plot of the Ackley function. Sine map, 32 particles per type.

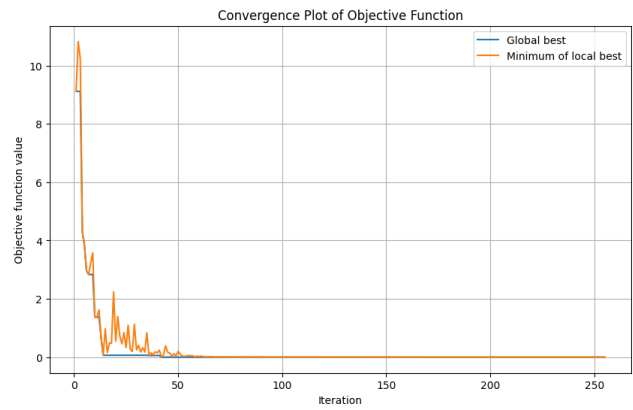


Figure 68: Convergence plot of the Ackley function. Singer map, 32 particles per type.

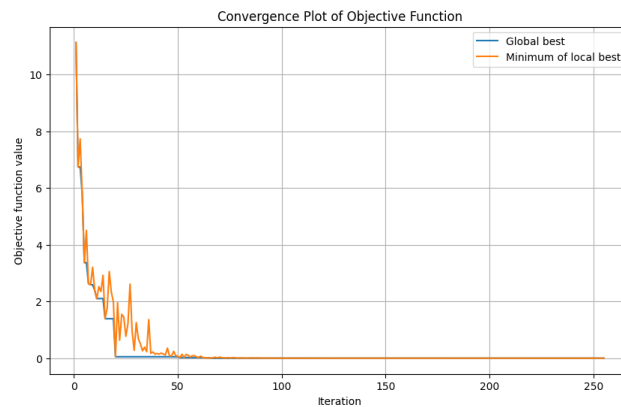


Figure 69: Convergence plot of the Ackley function. Sinusoidal map, 32 particles per type.